

THE FITTING PROCEDURE AND TRAINING OF THE bCCNN

Data splits, early stopping, RMSprop and the final fit

Study notes on the bCCNN strand of the research assignment

Prepared with the generative artificial intelligence tool Claude (Anthropic)
as study material for the BComHons research assignment
Department of Statistics and Actuarial Science
Stellenbosch University

Not for submission for assessment

October 2026

Note on the preparation of these notes

These notes were prepared with the generative artificial intelligence (AI) tool Claude (Anthropic) at the request of the student, as study material on the blended cross-classified neural network (bCCNN) strand of the research assignment. They explain the mathematics of the model, the code in the student's repository (Masters-Research-R) and the literature that the code follows.

The notes are not the student's original work and are not intended for submission for assessment. Section 2.6.3 of the Research Assignment Guide (Department of Statistics and Actuarial Science, 2026: 7-8) prohibits the use of AI tools to generate content that is submitted for assessment and requires a declaration of any use of such tools. For this reason the plagiarism declaration and the declaration on the use of artificial intelligence tools of the Department template are not included. The layout follows the technical requirements in Chapter 5 of the Guide and the Department template.

Abstract

This report explains the procedure by which the blended cross-classified neural network (bCCNN) for claims reserving is fitted in the code of the thesis: which data the cross-classified over-dispersed Poisson model and the network are fitted on, which data decide when gradient descent stops, which data are held back to judge the result, what the optimiser does at each step, which elements are random, and what the reported reserve is therefore a function of. Each step of the code is written as a formula and compared with the procedures of Gabrielli, Richman and Wüthrich, of Härkönen and of Al-Mudafer, Avanzi, Taylor and Wong.

The 50/50 split of the individual claims is shown to give two halves whose model parameters agree except for an intercept lower by $\log 2$, so that a network trained on one half can be validated directly on the other; the split measures how well the learned surface transfers to independent noise at the same cells, not extrapolation into the lower triangle. The steps of the optimiser are bounded by a fixed amount per coordinate, so that the number of steps limits how far the network can travel from the chain ladder. The number of steps is chosen as the exact minimum of the validation deviance over a run of 1 000 steps and is then used in a refit on the whole observed triangle; the alternative of keeping the early-stopped network of a rolling-origin partition is also described.

A replication of the procedure in plain numpy on a synthetic portfolio confirms the relation between the halves, an interior minimum of the validation curve and the bookkeeping of the tables produced by the code.

Key words: Claims reserving; Chain ladder; Over-dispersed Poisson model; Neural networks; Early stopping; Root mean square propagation; Validation; Back-testing

Table of contents

Note on the preparation of these notes.....	ii
Abstract.....	iii
List of tables.....	vi
List of figures.....	vii
List of abbreviations and/or acronyms.....	viii
CHAPTER 1 INTRODUCTION.....	1
1.1 INTRODUCTION.....	1
1.2 QUESTION ADDRESSED AND OBJECTIVES.....	1
1.3 SCOPE AND LIMITATIONS.....	2
1.4 HOW THE PROCEDURE WAS CHECKED.....	2
1.5 CHAPTER OUTLINE.....	2
CHAPTER 2 THE DATA AND THE CLAIMS SPLIT.....	3
2.1 INTRODUCTION.....	3
2.2 THE DATA AND ITS THREE PARTS.....	3
2.3 THE 50/50 SPLIT OF THE CLAIMS.....	4
2.3.1 Definition.....	4
2.3.2 Properties.....	5
2.3.3 Cells without payments in the training half.....	6
2.4 SUMMARY.....	6
CHAPTER 3 THE OPTIMISATION.....	7
3.1 INTRODUCTION.....	7
3.2 THE TRAINING PROBLEM.....	7
3.3 RMSPROP.....	7
3.3.1 The update rule as Keras implements it.....	7
3.3.2 Step size.....	8
3.4 RANDOM ELEMENTS AND THE SEED.....	9
3.5 SUMMARY.....	9
CHAPTER 4 EARLY STOPPING AND THE FINAL FIT.....	11

4.1	INTRODUCTION.....	11
4.2	EARLY STOPPING: CHOOSING THE NUMBER OF STEPS.....	11
4.2.1	The run and the rule.....	11
4.2.2	How the papers choose the step.....	12
4.2.3	The offset in Keras's reported loss.....	12
4.2.4	The decreases reported in Paper C's Table 3.....	13
4.2.5	The same rule on the rolling-origin partitions.....	13
4.3	THE FINAL FIT.....	13
4.3.1	Refit on the whole observed triangle.....	13
4.3.2	Keeping the early-stopped network.....	14
4.4	THE PROCEDURE AS ONE MAP.....	14
4.5	SUMMARY.....	15
	CHAPTER 5 OUTPUTS, BACK-TEST AND THE LITERATURE.....	16
5.1	INTRODUCTION.....	16
5.2	OUTPUTS OF THE PROCEDURE.....	16
5.3	THE TEST SET AND THE BACK-TEST.....	16
5.4	THE PROCEDURE IN THE PAPERS AND IN THE CODE.....	17
5.5	SUMMARY.....	18
	CHAPTER 6 IMPLEMENTATION AND VERIFICATION.....	20
6.1	INTRODUCTION.....	20
6.2	WHERE EACH STEP IS IN THE CODE.....	20
6.3	NUMERICAL CHECK.....	21
6.4	SUMMARY AND CONCLUSIONS.....	23
	References.....	25
	APPENDIX A LIST OF SYMBOLS.....	26

List of tables

Table 2.1 The parts of the data.....	3
Table 5.1 The fitting procedure in the three references and in the code.....	18
Table 6.1 Where each step of the procedure is implemented in the code.....	20
Table 6.2 The procedure on the synthetic halves (amounts in millions of the synthetic data).....	22
Table A.1 Symbols used in this report.....	26

List of figures

Figure 2.1 The claims split, the early-stopping run built on it and the refit.....	5
Figure 6.1 The early-stopping run on the synthetic halves.....	22

List of abbreviations and/or acronyms

bCCNN	Blended cross-classified neural network
ccODP	Cross-classified over-dispersed Poisson
LoB	Line of business
MLE	Maximum likelihood estimate
ODP	Over-dispersed Poisson
RMSprop	Root mean square propagation

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The blended cross-classified neural network (bCCNN) of Gabrielli, Richman and Wüthrich (2020), referred to as Paper C as in the code, embeds the cross-classified over-dispersed Poisson (ccODP) model of the chain ladder in a neural network, in the spirit of the combined actuarial neural networks of Wüthrich and Merz (2019). The ccODP is an over-dispersed Poisson (ODP) model with an accident-period and a development-period effect; the network takes its maximum likelihood estimates (MLE) as inputs, and gradient descent starts at the chain ladder. This report is the second of four companion explainers on the bCCNN in the thesis. They are cited as companion I, *The mathematics behind the bCCNN*; companion II, this report, *The fitting procedure and training of the bCCNN*; companion III, *The bCCNN as a whole: interpretation, evaluation and uncertainty*; and companion IV, *The time-aware split for the bCCNN*. The four documents share their notation.

Companion I describes the model, the loss, the gradients and the start. This report describes the procedure around them: which data the ccODP and the network are fitted on, which data decide when the gradient descent stops, which data are held back to judge the result, what the optimiser does at each step, what is random, and what the fitted network is therefore a function of. It follows `bccnn_calibrate()` and the functions it calls, step by step, and writes each step as a formula. The code explained is `triangle_sets()` in `R/triangles.R`; `fit_bccnn()`, `bccnn_validation()`, `loss_decrease()`, `bccnn_phi()` and `bccnn_calibrate()` in `R/nn_models.R`; `bccnn_tables()` in `R/tables.R`; and the script `analysis/03_bCCNN/bCCNN fit.R`. It is read against Paper C, Härkönen (2021), a master's thesis that applies the bCCNN, and Al-Mudafer, Avanzi, Taylor and Wong (2021).

The default procedure in `config.yml` chooses the number of steps on rolling-origin partitions (validation: `rolling_origin`); that split is described in full in companion IV, and here it enters only where it plugs into the same procedure (Section 4.2.5). The alternative, Paper C's 50/50 split of the claims (validation: `claims_split`), is described in full here (Section 2.3), because it is the procedure of Paper C and of Härkönen (2021).

1.2 QUESTION ADDRESSED AND OBJECTIVES

The question addressed is what the reserve reported by the bCCNN is a function of, once the model of companion I is wrapped in the fitting procedure of the code. The objectives are to state, for each step of the procedure, which cells and which payments it uses; to derive the properties of the 50/50 split of the claims on which the validation rests; to describe the update rule of root mean square propagation (RMSprop) as Keras implements it, and the bound it places on the distance travelled from the chain ladder; to state the

stopping rule, the transfer of the chosen number of steps to the final fit and the outputs and back-test that follow; and to compare the procedure with the three references.

The procedure matters because the number of gradient-descent steps is the one quantity the procedure tunes: with zero steps the bCCNN is the chain ladder, and every departure from the chain ladder in the reported reserve is produced by the steps that the procedure allows.

1.3 SCOPE AND LIMITATIONS

The report explains the code as it stands; it does not run Keras. Quotations from Paper C and from Al-Mudafer *et al.* are taken from the literature notes in the repository (thesis/Hyperparameter search), which cite Paper C through its SSRN working-paper version (Gabrielli, Richman and Wüthrich, 2018) and both papers by PDF page; the page numbers given are those of the notes. These notes have not re-checked the two papers themselves. Quotations from Härkönen (2021) were checked against the thesis and are cited by its printed page numbers. The numerical check of Chapter 6 uses synthetic data with the dimensions of the data of the thesis; its numbers are not a statement about the simulated portfolio of the thesis.

1.4 HOW THE PROCEDURE WAS CHECKED

The procedure of Sections 2.3 to 5.2 was run in plain numpy (no Keras) on a synthetic portfolio built as two independent halves with the dimensions of the data of the thesis, with Keras's RMSprop rule and its inverted form of dropout (Srivastava, Hinton, Krizhevsky, Sutskever and Salakhutdinov, 2014) re-implemented from their definitions. Section 6.3 reports the run: the ccODP parameters of the two halves relate as Section 2.3.2 predicts, the validation curve has an interior minimum, and the refit reproduces the bookkeeping of `bccnn_calibrate()` and `bccnn_tables()` row by row.

1.5 CHAPTER OUTLINE

Chapter 2 describes the data, its three parts and the 50/50 split of the claims with its properties. Chapter 3 states the training problem, the RMSprop update and the random elements of the procedure. Chapter 4 describes the early-stopping run that chooses the number of steps, the final fit that uses it, and the whole procedure as one map. Chapter 5 lists the outputs, defines the back-test against the simulated lower triangle and compares the procedure with the three references. Chapter 6 locates each step in the code, reports the numerical check and draws the conclusions. The references and a list of symbols follow.

CHAPTER 2

THE DATA AND THE CLAIMS SPLIT

2.1 INTRODUCTION

Chapter 1 set out the question of what the reported reserve depends on. The first part of the answer is the data. This chapter describes the three parts into which the code divides the data, and then the 50/50 split of the individual claims that Paper C uses to validate the network, with the properties that make the split usable and the masking of periods without payments that the code adds.

2.2 THE DATA AND ITS THREE PARTS

`triangle_sets(transactions, n_dev = 20, claims)` turns the payment transactions of the simulator into 20×20 incremental squares (rows are accident years, columns are development years $1, \dots, 20$; payments after development year 20 are left out, see companion I, Section 1.2). The script writes five of them to `data/interim`. For the procedure they form the three parts of Table 2.1.

Table 2.1 **The parts of the data**

Part	Matrix	Cells	Used for
The observed triangle D	<code>sets\$upper</code>	O , 210 cells; future cells NA	The ccODP of the final fit and the training of the final network (Section 4.3); the in-sample loss (Section 5.2)
The two halves D^T, D^V	<code>sets\$train, sets\$vali</code>	O each	Choosing t^* under <code>claims_split</code> (Sections 2.3 and 4.2); never used by the final fit
The test set T	<code>sets\$test</code>	F , 190 cells; observed cells NA	The back-test only: the true reserves and the out-of-sample loss (Section 5.3); never used to fit or to choose t^*

Under `rolling_origin` the halves are not used; the partitions are cut out of D instead (companion IV, Section 2.3).

All amounts are divided by $u = 10^6$ (`dat_cfg$scale`) before anything is fitted; companion I (Section 2.9) shows that this only shifts the ccODP intercept.

2.3 THE 50/50 SPLIT OF THE CLAIMS

2.3.1 Definition

The upper triangle has 210 cells and the ccODP needs every accident period and every development period to have cells, so holding back cells of D for validation is not an option in the view of Paper C (Section 3.3.1 and Listing 2 there): "one of the reasons being that we only have $(I + 1)I/2 = 78$ observations in the upper triangle" (Gabrielli *et al.*, 2018: 11), on their 12×12 triangles. The split is made one level below the triangle, on the individual claims, with halves balanced by line of business (LoB) and by accident year: "we choose training and validation sets by partitioning on the individual claims level such that both sets have (approximately) the same size (on a LoB level and on an accident year level)" (Gabrielli *et al.*, 2018: 11). Härkönen does the same: "we have divided the individual claims data into training and validation data that are approximately the same size (50/50) since the model parameters from the ODP model are sensitive to the volume of the data and these are the input weight to our neural network models" (Härkönen, 2021: 27).

`triangle_sets()` implements Paper C's Listing 2. The claims are ordered by accident period and, within it, by claim number, and are allocated alternately in that order,

$$h(\text{claim } r) = 1 + ((r - 1) \bmod 2) \in \{1, 2\}, r = 1, 2, \dots, \quad (2.1)$$

and the payments of each half are aggregated into a square of its own. With h_r the half of the claim that payment r belongs to,

$$\begin{aligned} y_{i,j}^T &= \sum_{r: h_r=1, i_r=i, j_r=j} \text{payment}_r, \\ y_{i,j}^V &= \sum_{r: h_r=2, i_r=i, j_r=j} \text{payment}_r, \\ y_{i,j}^T + y_{i,j}^V &= y_{i,j}. \end{aligned} \quad (2.2)$$

The code asserts the last identity (`train_full + vali_full == full`). Both halves are then cut to their observed triangles (`upper()`), D^T and D^V , and `n_claims` records the number of claims per accident period and half. Figure 2.1 shows the claims split together with the early-stopping run built on it (Section 4.2) and the refit (Section 4.3).

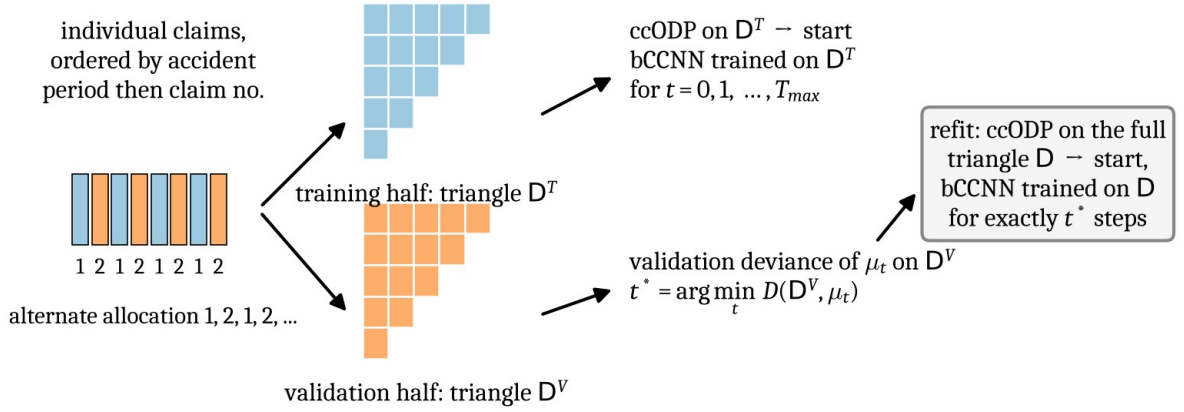


Figure 2.1 The claims split, the early-stopping run built on it and the refit

2.3.2 Properties

(a) *The halves are balanced by accident period.* Because the ordering is by accident period and the allocation alternates along it, the two halves of accident period i differ by at most one claim: $|n_i^T - n_i^V| \leq 1$ for every i . This is what Paper C means by halves of "(approximately) the same size (on a LoB level and on an accident year level)" (Gabrielli *et al.*, 2018: 11).

(b) *The ccODP parameters carry over from one half to the other, except the intercept.* Suppose the claims of an accident period are exchangeable, so that each half is a random half of the portfolio, and that the whole portfolio follows the ccODP of companion I, (2.1). Then the cells of each half have mean $\mu_{i,j}/2$ and are themselves over-dispersed Poisson (the scaled-Poisson representation is closed under this thinning), with the same ϕ . On the log scale,

$$\log(\mu_{i,j}/2) = (c - \log 2) + \alpha_i + \beta_j, \quad (2.3)$$

so the MLE on one half estimates the same α and β as the full triangle and an intercept lower by $\log 2 = 0.693$. The MLEs on the two halves are therefore estimates of the same effects, and the means fitted on D^T are, in expectation, the means of D^V . This is why the validation loss of Section 4.2.1 can compare the network trained on D^T directly with the cells of D^V , with no rescaling; and why Paper C and Härkönen insist on halves of equal size (with unequal halves the intercept would differ by the logarithm of the volume ratio and the comparison would need an offset). In the numerical check the two intercepts differ by 0.676 against $\log 2 = 0.693$, the accident effects agree within 0.06 and the development effects within 0.41 (the largest gaps are in the late, sparse development periods); see Section 6.3.

(c) *What the validation loss measures.* Both halves contain the same accident periods, development periods and calendar periods. The validation loss of a network trained on D^T is its deviance on an independent copy of the *same upper triangle*: it measures how well the surface the network has learned over $(\hat{\alpha}_i, \hat{\beta}_j)$ transfers to independent noise at the *same* (i, j) pairs. It does not measure extrapolation to

the lower triangle, whose (i, j) pairs are in neither half. Paper C saw this gap in the favourable direction: "the neural network boosting improvement has been successful in all LoBs here, even though the initial out-of-sample validation analysis in the upper triangles of Section 3.3.2 has not been suggesting this for the LoBs 3 and 6" (Gabrielli *et al.*, 2018: 14). The rolling-origin split of companion IV is the time-aware alternative.

(d) *Sensitivity to the split.* Härkönen found "that the machine learning models can be sensitive to the division of training and validation sets" (Härkönen, 2021: 33), refitted with the halves swapped (the 'VT' rows of her Table 3) and found biases of opposite sign for several models, "which gives rise to selecting the average outstanding reserve as the prediction" (Härkönen, 2021: 34). Only one allocation, (2.1), is used in the code.

2.3.3 Cells without payments in the training half

A development period in which the training half has no payments at all has ccODP effect $-\infty$ (companion I, Section 2.10), and the means of the network there are ≈ 0 , so a validation cell in that period would contribute $y \log(y/\mu) \rightarrow \infty$ to the deviance for any $y > 0$. `bccnn_validation()` therefore drops from the validation matrix every row in `odp$zero_origin` and every column in `odp$zero_dev` (sets them to NA): the validation deviance is taken over the cells of D^V whose accident and development periods have payments in D^T . It does so through the helper `mask_zero_periods()`, which the rolling-origin partitions also use (companion IV, Section 3.3). On the annual triangle this affects at most the last development periods.

This masking is the code's own. Neither Paper C nor Härkönen discusses periods without payments: with their large portfolios and claims splits balanced by accident year, every period has payments in both halves (companion I, Section 2.10).

2.4 SUMMARY

The data enter the procedure in three parts: the observed triangle, which trains the final model; the two halves of the claims, which only choose the number of steps; and the true lower triangle, which only judges the result. The alternating allocation of the claims balances the halves by accident period, and under exchangeable claims the two halves share the accident and development effects of the ccODP, with an intercept lower by $\log 2$. The validation loss on the second half is therefore directly comparable with the fit on the first, but it measures transfer to independent noise at the same cells, not extrapolation. Validation cells in periods without payments in the training half are masked. Chapter 3 turns to what is done with the training cells: the loss, the optimiser and the random elements.

CHAPTER 3

THE OPTIMISATION

3.1 INTRODUCTION

Chapter 2 fixed the cells on which the network is trained and validated. This chapter states the training problem on those cells, the RMSprop update that Keras applies to it and the consequences of that update for the size of each step, and the random elements that the seed controls.

3.2 THE TRAINING PROBLEM

The network is trained on the N cells of one observed triangle, D (final fit) or D^T (validation run), by minimising Keras's mean Poisson loss ((3.8) of companion I), which is the Poisson deviance up to a constant ((3.9) of companion I):

$$\ell(\theta) = \frac{1}{N} \sum_{(i,j) \in O} (\mu_{i,j}(\theta) - y_{i,j} \log \mu_{i,j}(\theta)), N = 210. \quad (3.1)$$

`fit_bccnn()` passes the inputs of the training cells (`x_fit`) and their payments (`y_fit`) to `fit()` with `batch_size = batch_size % || % nrow(y_fit)`. With `batch_size: ~` in `config.yml` the batch is the whole triangle, so one Keras epoch is one evaluation of the gradient of (3.1) and one parameter update: epochs and gradient-descent steps coincide, as in Paper C, whose Figure 2 has 'gradient descent iteration' on its axis, and as the comment in the code states (full batch (1 epoch = 1 step)). If a smaller `batch_size` were set, an epoch would consist of $\lceil N / \text{batch} \rceil$ steps on reshuffled mini-batches and the history below would be recorded once per epoch. The run has no patience and no restoration of best weights inside Keras: `fit()` is asked for exactly `epochs` epochs, and the choice of the step happens afterwards in R (Section 4.2).

After every step the callback records the row of Table 3.2 of companion I: the Keras loss of that epoch as a deviance (`train_dropout`), the deviance without dropout on the training cells (`train`) and on each tracked matrix (`vali`, `test`, `truth`), and the predicted sums over the tracked cells (`*_pred`). The row with `epoch = 0` is the ccODP start; its `train_dropout` is NA because Keras has not reported anything yet.

3.3 RMSPROP

3.3.1 The update rule as Keras implements it

`optimizer_rmsprop(learning_rate = 0.001, rho = 0.9, epsilon = 1e-7)` (momentum 0, not centred) keeps, for every coordinate ϑ of θ , an exponential moving average of the squared gradient and divides the gradient by its root:

$$\begin{aligned}
g_t &= \frac{\partial \ell}{\partial \vartheta}(\theta_{t-1}), v_t = \rho v_{t-1} + (1 - \rho) g_t^2, v_0 = 0, \\
\vartheta_t &= \vartheta_{t-1} - \eta \frac{g_t}{\sqrt{v_t + \epsilon}}.
\end{aligned} \tag{3.2}$$

Härkönen's Algorithm 2 (Härkönen, 2021: 23) is the version of the same rule in Section 8.5.2 of Goodfellow, Bengio and Courville (2016), with the stabiliser inside the root, $\Delta \vartheta = -\eta g / \sqrt{\delta + v}$ and $\delta = 10^{-6}$; Keras puts ϵ outside the root. The difference is immaterial unless v_t is of the order of ϵ^2 . Härkönen's reasons for RMSprop are those of the textbook: it "has been empirically shown to be an effective method for training a deep neural network", and the decay ρ "discards the information from past extremes so that it can converge rapidly after finding a convex bowl" (Härkönen, 2021: 22).

3.3.2 Step size

Three facts about (3.2) explain the shape of the loss curves.

The first step is sign-like. Since $v_1 = (1 - \rho) g_1^2$, the first update is $\Delta \vartheta_1 = -\eta g_1 / (\sqrt{1 - \rho} |g_1| + \epsilon) \approx -(\eta / \sqrt{1 - \rho}) \text{sign}(g_1) = \mp 0.00316$ for every coordinate whose gradient exceeds $\epsilon / \sqrt{1 - \rho} \approx 3 \times 10^{-7}$ in size. At the start only the ten coordinates of B have such a gradient (companion I, Section 4.5), so step 1 moves exactly those ten by ± 0.00316 (companion I, Section 4.6 and (4.6)).

Every step is bounded. Since $v_t \geq (1 - \rho) g_t^2$ always, $|g_t| / \sqrt{v_t} \leq 1 / \sqrt{1 - \rho} = 3.16$, hence

$$|\vartheta_t - \vartheta_0| \leq \frac{\eta}{\sqrt{1 - \rho}} t = 0.00316 t \text{ for every coordinate and every } t. \tag{3.3}$$

After $t^* = 300$ steps no weight can have moved by more than 0.95; after 1 000 steps, by more than 3.16. In the steady state of a slowly varying gradient, $v_t \approx g_t^2$ and the step is $\approx \eta = 0.001$ per coordinate. The number of steps is therefore a hard budget on how far the network can travel from the chain ladder, which is what makes early stopping a regulariser here (companion III, Section 3.2).

The scale of the gradient hardly matters. Multiplying the loss by a constant multiplies g_t and $\sqrt{v_t}$ by the same constant, so (3.2) is unchanged except for ϵ . This is why the factor $2N$ between Keras's loss and the deviance, and the unit of the payments (companion I, Section 2.9), do not change the training path in any material way; the separate explainer on the neural network chain ladder in the repository shows that plain stochastic gradient descent behaves differently.

The window of the moving average is about $1/(1 - \rho) = 10$ steps: v_t remembers the squared gradients of roughly the last ten steps, so under dropout, where the gradient changes from step to step, the step size adapts to the recent noise level.

3.4 RANDOM ELEMENTS AND THE SEED

`bccnn_model()` calls `clear_session()` and then `set_random_seed(seed)` (the helper of Keras, which seeds R, numpy and the backend) before it builds the model, so two things are reproducible given s :

- The initial hidden-layer weights. They are Glorot-uniform, $W_{r,q}^{(h)} \sim U(-\lambda_h, \lambda_h)$ with $\lambda_h = \sqrt{6/(q_{h-1} + q_h)}$, that is $\lambda_1=0.522$, $\lambda_2=0.414$ and $\lambda_3=0.490$, with zero biases. The output neuron and the embeddings are then overwritten with the ccODP start, so the only random part of θ_0 is the three hidden layers;
- The dropout masks. One Bernoulli mask per hidden layer per step is drawn from the same seeded generator.

Because the seed is set every time a model is built, every fit in one run of `bccnn_calibrate()` (the validation run, each rolling-origin partition, the refit) starts from the same random hidden-layer weights; fits with the same number of training cells also see the same sequence of dropout masks. So, given the seed, the whole procedure is deterministic, and t^* found on D^T is applied to a refit whose random features are the same ones the validation run had.

What the seed changes is substantial. Härkönen observes that "the neural network models are quite sensitive for the choice of seed as well" (Härkönen, 2021: 36); her results use seed 75. She refitted the double network for 20 seeds, found biases that vary by several percentage points, and recommends fitting "several models with different seeds and then choose the average prediction" (Härkönen, 2021: 36-37, 53). Al-Mudafer *et al.* average the error of 2 T differently initialised runs when tuning (Al-Mudafer *et al.*, 2021: 16), and ensemble five fits of the final model (Al-Mudafer *et al.*, 2021: 17). The code fits one seed (`cfg$seed = 2026`); a seed study or a nagging average is a change to the script, not to the procedure (companion III, Section 5.5).

A remark on the inputs. The hidden layers read the raw MLEs $(\hat{\alpha}_i, \hat{\beta}_j)$, not standardised values (Paper C, Listing 4, does the same). The development effects of a short-tailed triangle span several units ($\hat{\beta}_j$ is very negative in the late development periods), so with $\lambda_1=0.52$ some first-layer pre-activations $b + W^{(1)T} z^{(0)}$ exceed 2 in size there, where $\tanh$ is nearly flat and its derivative $1 - z^2$ is nearly zero. For those cells the gradient into the hidden layers is small: the network learns its features mostly from the early development periods and treats the late ones through the skip connection. This is a property of the design, not a fault, but it is worth knowing when reading the relative-difference heat map.

3.5 SUMMARY

The network is trained by full-batch minimisation of Keras's mean Poisson loss, so that an epoch is one gradient-descent step and every step is recorded. RMSprop makes the first step sign-like, bounds every

step by 0.00316 per coordinate, and is insensitive to the scale of the loss, so that the number of steps is a hard budget on the distance from the chain ladder. The seed fixes the random hidden-layer weights and the dropout masks; given the seed the procedure is deterministic, but the seed itself moves the result by several percentage points in Härkönen's study. Chapter 4 describes how the number of steps is chosen and how it is used in the final fit.

CHAPTER 4

EARLY STOPPING AND THE FINAL FIT

4.1 INTRODUCTION

Chapter 3 showed that the number of RMSprop steps controls how far the network moves from the chain ladder. This chapter describes how the code chooses that number on the validation half, how the three references choose it, how Keras's reported loss is offset against the recorded deviances, which decreases are reported, and how the rule carries over to the rolling-origin partitions. It then describes the two ways of building the final model and writes the whole procedure as one map.

4.2 EARLY STOPPING: CHOOSING THE NUMBER OF STEPS

4.2.1 The run and the rule

Paper C asks: "A crucial question that has not been touched, yet, is how many iterations of the gradient descent algorithm we should perform" (Gabrielli *et al.*, 2018: 11). The answer, in the terms of Goodfellow *et al.* (2016: 248-249, Algorithm 7.2), is to find the number of steps on a training and validation split and then retrain on all data for that many steps. `bccnn_validation()` proceeds in three steps.

First, the ccODP is fitted on the training half: $\hat{\vartheta}^T = (\hat{c}^T, \hat{\alpha}^T, \hat{\beta}^T)$ from D^T (`fit_odp_glm(train)`), and the start θ_0^T is built from it ((3.7) of companion I).

Second, $T_{max} = 1\,000$ full-batch RMSprop steps are run on the cells of D^T with dropout, and after every step the validation deviance without dropout is recorded,

$$D_V(t) = D(y^V, \mu(\theta_t^T); O \mid \text{masked}), t = 0, 1, \dots, T_{max}, \quad (4.1)$$

and likewise $D_T(t)$ on D^T and the Keras loss $\ell_t^{drop}(\text{train_dropout})$.

Third, the step with the lowest validation deviance over the whole run is chosen:

$$t^* = \arg \min_{0 \leq t \leq T_{max}} D_V(t). \quad (4.2)$$

Ties go to the earliest step (which.min). The value $t^* = 0$ is allowed and means that the chain ladder cannot be improved on the validation half: the final fit then takes zero steps and returns the chain ladder.

There is no patience and no smoothing in (4.2): the whole run of T_{max} steps is always completed, and the minimum is read from the recorded curve, which is computed without dropout and is therefore smoother than Keras's own loss. `config.yml` can override the result (`epochs: <number>`), in which case `bccnn_tables()` reports override (`config epochs`) and the decreases of Section 4.2.4 are evaluated at that step.

4.2.2 How the papers choose the step

The same curve is read differently in the three references, and the differences matter for how t^* should be interpreted.

Paper C. Paper C reads the curve by eye from its Figure 2: "If we focus on the out-of-sample losses (red color), we notice that they seem to have a minimum after roughly 300 iterations (dotted vertical line). This suggests that the first 300 steps of the algorithm improve the predictive power of the bCCNN model, and [...] after that the algorithm starts to over-fit to the observations" (Gabrielli *et al.*, 2018: 12-13). It then uses the one number for every line of business: "we decide to run the gradient descent algorithm on the full data DI|m for 300 iterations [...] for simplicity, we use 300 iterations for all LoBs" (Gabrielli *et al.*, 2018: 13), including LoBs where the validation curve showed no gain.

Härkönen. Härkönen runs 10 000 epochs and takes the minimum of a central moving average: "The number of epochs is then chosen by a simple central moving average with window size 100" (Härkönen, 2021: 32). Her Table 2 shows how spread the results are: 274 to 9 899 epochs for the payment (amount) part of the simple network across the six LoBs, up to 9 950 for its count part, and about 7 000 for most LoBs of the double network (Härkönen, 2021: 33). She notes validation curves that are "a straight line through all 10 000 epochs" (no gain) and a validation error that "first drops and then starts to increase again" (Härkönen, 2021: 33), the textbook over-fitting shape.

Al-Mudafer et al. Al-Mudafer *et al.* use the patience of Keras on a rolling-origin validation set: "training was only stopped when it did not hit new lows in the last 1000 epochs [...] a lower patience than 1000 would sometimes prematurely stop training" (Al-Mudafer *et al.*, 2021: 16), with a cap of 10 000 epochs.

The code. The code uses (4.2): the exact minimum over a run of 1 000 steps. It is the least smoothed of the four rules. Under dropout the parameters jitter from step to step, so $D_V(t)$ has step-to-step noise of its own even though it is computed without dropout; the exact minimum can land on a lucky step. If that is a concern, a moving-average minimum in the spirit of Härkönen is a one-line change on the recorded history (h\$vali), because the whole curve is kept.

4.2.3 The offset in Keras's reported loss

Keras computes the loss of epoch e on the forward pass that precedes the update of that epoch, so the number it reports for epoch e is the dropout loss at θ_{e-1} , while train and vali of row e are evaluated at θ_e . The comment on `loss_decrease()` in the code records this: "Keras's training loss of epoch e is computed before that epoch's update, i.e. after $e - 1$ steps". Hence `train_dropout` at epoch 1 is the dropout loss of the ccODP start, and the dropout loss that belongs to step t sits in row $t + 1$.

4.2.4 The decreases reported in Paper C's Table 3

`loss_decrease(h, step, window = 10)` returns three relative decreases from the start to step, matching the rows of Paper C's Table 3:

$$\begin{aligned}\Delta_T &= 1 - \frac{D_T(t^*)}{D_T(0)}, \Delta_V = 1 - \frac{D_V(t^*)}{D_V(0)}, \\ \Delta_T^{drop} &= 1 - \frac{\frac{1}{2w+1} \sum_{e=t^*+1-w}^{t^*+1+w} \ell_e^{drop}}{\ell_1^{drop}}, w = \min(10, t^*),\end{aligned}\tag{4.3}$$

all with the losses expressed as deviances. The third one averages Keras's noisy dropout loss over ± 10 epochs around row $t^* + 1$ (the offset of Section 4.2.3) and compares it with row 1, the dropout loss of the start; at $t^* = 0$ it is set to 0. `bccnn_tables()` prints them in per cent. Δ_V also sets the bCCNN dispersion (Section 5.2).

4.2.5 The same rule on the rolling-origin partitions

Under validation: rolling_origin the run of Section 4.2.1 is made on each partition: the ccODP is fitted on the training cells of the partition, the network is trained on them for T_{max} steps, and $D_V(t)$ is the deviance on the validation cells of the partition (its latest calendar periods). The step used for the final fit is the t^* of the *final* partition, the one whose triangle is all of D ; the test partitions, which end earlier, serve to score the procedure out of time. Validation cells in periods with no payments in the training cells of a partition are masked exactly as under the claims split. Everything from Section 4.3 on is the same; companion IV has the details and the counts (Sections 2.3 and 3.3 there).

4.3 THE FINAL FIT

4.3.1 Refit on the whole observed triangle

With t^* chosen, the reported model is built from scratch on the whole observed triangle (`final_fit: refit`), as in Paper C and Härkönen: "the models are fitted again on the entire upper triangle (training and validation data) and predictions are made on the lower triangle" (Härkönen, 2021: 33); "we decide to run the gradient descent algorithm on the full data [...] for 300 iterations" (Gabrielli *et al.*, 2018: 13). `bccnn_calibrate()` proceeds as follows:

- i) It fits $\hat{\vartheta} = (\hat{c}, \hat{\alpha}, \hat{\beta})$ from D (`fit_odp_glm(sets$upper)`), the chain ladder of the whole portfolio.
- ii) It builds the start θ_0 from $\hat{\vartheta}$ with seed s and runs exactly t^* full-batch RMSprop steps on the 210 cells of D , with dropout (`fit_bccnn(odp, epochs = steps, track = list(truth = truth))`). The true lower triangle is tracked during this fit for the plot of the out-of-sample loss by step, but it influences nothing: the number of steps was fixed before.

- iii) It computes the means $\mu^{bCCNN} = \mu(\theta_{t^*})$ over the whole square, the reserves from their future cells and the in-sample deviance on O , and saves the model to `models/bccnn_annual.keras`.

What is transferred from the validation run to the refit is the *count* t^* , not a state: the refit starts from a different ccODP ($\hat{c}$ instead of $\hat{c}^T$, and the $\hat{\alpha}, \hat{\beta}$ of the full triangle) and sees different payments (twice the volume). The assumption behind the transfer is that the loss surface of the full triangle is, up to the volume, the surface the validation run explored, so that the same number of steps is the right distance to travel. Property (b) of Section 2.3.2 is the argument for this under the claims split (same effects, intercept shifted by $\log 2$, and RMSprop insensitive to the factor 2 in the gradients); it is an assumption all the same, and Paper C's use of one t^* for all six LoBs stretches it further than the code does.

4.3.2 Keeping the early-stopped network

The alternative (`final_fit: partition`), available for the rolling origin only, is that of Al-Mudafer *et al.*: the reported model is the network of the final partition at its best step. `bccnn_rolling_origin()` keeps the mean square after every step of that partition (`keep_mu_final`), and `bccnn_calibrate()` takes $\mu^{bCCNN} = \mu^{(t^*)}$ from `val$mu_path[[steps + 1]]`. No second network is trained and the validation cells never enter the training; in exchange the model was trained on 177 of the 210 cells rather than all of them, and its loss on the whole observed triangle mixes in-sample cells and the 33 validation cells (`bccnn_tables()` labels that row accordingly and keeps deviance on the training cells apart from `deviance_observed`).

4.4 THE PROCEDURE AS ONE MAP

Let $T_t^s(\theta, S)$ denote t full-batch RMSprop steps with dropout and seed s , from θ , on the cells of the triangle S (the loss (3.1) on S), and $\theta_0(\hat{\vartheta})$ the start of (3.7) of companion I built from a ccODP fit $\hat{\vartheta}$. Then, under the claims split,

$$\begin{aligned} t^* &= \arg \min_{0 \leq t \leq T_{\max}} D\left(y^V, \mu\left(T_t^s\left(\theta_0(\hat{\vartheta}(D^T)), D^T\right)\right); O\right), \\ \hat{\theta} &= T_{t^*}^s\left(\theta_0(\hat{\vartheta}(D)), D\right), \\ \hat{R}^{bCCNN} &= \sum_F \mu_{i,j}(\hat{\theta}). \end{aligned} \tag{4.4}$$

The reported reserve is a function of the observed triangle D , of the allocation (2.1) of the claims (through D^T , D^V and t^*), of the seed s , and of the fixed hyperparameters $(q, p, \eta, \rho, \epsilon, T_{\max})$. Under the rolling origin, the first line is replaced by the run on the final partition ((3.2) and (3.5) of companion IV) and the allocation drops out; under `final_fit: partition`, the second line is replaced by the partition's own θ_{t^*} .

4.5 SUMMARY

The number of steps t^* is the exact minimiser of the validation deviance over a complete run of 1 000 steps on the training half, with no patience and no smoothing; Paper C reads its minimum by eye and uses one value for all LoBs, Härkönen smooths with a moving average and Al-Mudafer *et al.* use a long patience. Keras's reported loss lags the recorded deviances by one row, which the decreases of Paper C's Table 3 take into account. Under the default refit only the count t^* is transferred to a new fit on the whole observed triangle, which rests on the assumption that the full triangle presents the same loss surface up to volume; the alternative keeps the early-stopped network of the final rolling-origin partition. The whole procedure is a deterministic map from the triangle, the allocation of the claims, the seed and the hyperparameters to the reserve. Chapter 5 describes what the procedure reports and how the result is judged.

CHAPTER 5

OUTPUTS, BACK-TEST AND THE LITERATURE

5.1 INTRODUCTION

Chapter 4 ended with the fitted network and its reserve. This chapter lists the tables and figures that the code produces from the fit, defines the back-test against the simulated lower triangle, and sets the whole procedure of the code beside the procedures of Paper C, Härkönen and Al-Mudafer *et al.*

5.2 OUTPUTS OF THE PROCEDURE

`bccnn_tables(res, sets)` writes the following tables, in units of u , after Paper C's Tables 2 to 4.

The results table (Paper C's Tables 2 and 4, one column) contains the true reserve $R = \sum_F y_{i,j}$; the chain-ladder reserve $\hat{R}^{cc}$ and the bCCNN reserve $\hat{R}^{bCCNN}$ ((5.1) of companion I); their biases $\hat{R} - R$ and $100(\hat{R}/R - 1)$; the in-sample deviances $D(y, \hat{\mu}^{cc}; O)$ and $D(y, \mu^{bCCNN}; O)$; the out-of-sample deviances on the true lower triangle $D(y, \cdot; F)$; the dispersions $\hat{\phi}_D$ and

$$\hat{\phi}^{bCCNN} = \hat{\phi}_D (1 - \max(0, \Delta_V)) \quad (5.1)$$

(Paper C, Section 3.3.2; `bccnn_phi()`); the number of steps t^* ; the payments after development period n (`tail_o`, in no model); and the unit.

The validation table (Paper C's Table 3) contains T_{max} , t^* , the steps used, $D_T(0)$ and $D_T(t^*)$, $D_V(0)$ and $D_V(t^*)$, and the three decreases (4.3) in per cent.

The other tables. `by_origin` gives, per accident period, the latest cumulative payments, the true outstanding payments, both reserves and both biases, with a total row; `odp_parameters`, `mu_ccODP`, `mu_bCCNN`, `history_validation` and `history_fit` hold the ccODP effects, the two mean squares and the two recorded curves; and under `rolling_origin` the `rolling_origin` table of companion IV (Section 3.5 there) is added.

The figures of the script are the loss curves of the validation run with t^* marked (after Paper C's Figure 2), the loss curves of the final fit (in-sample and on the true lower triangle), the relative difference $\mu^{bCCNN}/\hat{\mu}^{cc} - 1$ (after Paper C's Figure 5), the Pearson residuals of both models on the lower triangle on a common colour scale (after Paper C's Figure 7), the cumulative development factors (after Paper C's Figure 8), the bias by accident period, and the partition tiles (after Figures 3 and 9 of Al-Mudafer *et al.*).

5.3 THE TEST SET AND THE BACK-TEST

The lower triangle T enters only after the fit. Two numbers are computed from it for each model:

$$\begin{aligned}
\text{bias} &= \hat{R} - R = \sum_F (\mu_{i,j} - y_{i,j}), \\
\text{out-of-sample loss} &= D(y, \mu; F).
\end{aligned} \tag{5.2}$$

The bias is the quantity a reserving model is judged on; the deviance on F is the same criterion the model was trained by, evaluated where it was not trained, and it is sensitive to cell-level errors that cancel in the bias. Payments after development period n are outside both, and are listed separately so that the reserve to development period n is compared with the truth to development period n . Both numbers are known only because the data are simulated; the procedure never uses them to choose anything (the truth column of the history of the final fit is recorded after the number of steps was fixed).

5.4 THE PROCEDURE IN THE PAPERS AND IN THE CODE

Table 5.1 sets the procedure of the code beside those of the three references. The double network of Härkönen (2021) follows Gabrielli (2020). The entries for Paper C and Al-Mudafer *et al.* rest on the quotations in the literature notes in the repository (thesis/Hyperparameter search).

Table 5.1 The fitting procedure in the three references and in the code

Aspect	Paper C	Härkönen (2021), simple network	Al-Mudafer <i>et al.</i> (2021)	The code, config.yml defaults
Data	Six LoBs of simulated claims, 12×12 triangles, 78 cells, amounts in thousands of Swiss francs	The same simulator with reporting and payment delay, plus Folksam data; counts and amounts in separate networks	40×40 quarterly triangles, mixture density network (not a bCCNN)	One 20×20 annual triangle, 210 cells, millions
Network	(20, 15, 10) tanh neurons, 10% dropout, fixed embeddings, trainable skip weight	The same; also a double network ((30, 25) neurons, 20% dropout, L^2 penalty, batch normalisation)	Mixture density network, tuned	That of Paper C, bccnn: model:
Loss and optimiser	Poisson deviance, RMSprop, full batch	Poisson deviance, RMSprop (Keras)	Negative log-likelihood, Adam	Keras poisson, RMSprop (0.001, 0.9, 10^{-7}), full batch
Validation data	50/50 split of the claims	50/50 split of the claims; also swapped	Rolling-origin partitions, latest calendar periods	Rolling origin (default) or claims split
Stopping rule	By eye, about 300; the same for all LoBs	Minimum of a 100-epoch moving average over 10 000 epochs	Patience 1 000, cap 10 000	Exact minimum over 1 000 steps; override possible
Final model	Refit on all claims, 300 steps	Refit on the whole upper triangle for the chosen epochs	Model of the final partition, ensemble of 5 seeds	Refit (default) or the network of the final partition
Dispersion	That of the ccODP, reduced by the validation decrease	Not computed for the networks (parameters exceed observations)	From the predicted density	The rule of Paper C, (5.1)
Randomness	One run per LoB, dropout found stable at 10%	Seed 75; 20-seed study; averaging recommended	$2T$ runs in tuning; 5-model ensemble	One seed, 2026
Back-test	True lower triangle of the simulation	True lower triangle; mean squared error of prediction by bootstrap	Test partitions and the actual lower triangle	True lower triangle; test partitions under the rolling origin

5.5 SUMMARY

The code reports Paper C's tables of results and of validation decreases, the reserves and biases by accident period, the ccODP effects, both mean squares and both recorded curves, together with the figures after Paper C and Al-Mudafer *et al.* The bCCNN dispersion is the ccODP dispersion reduced by the validation decrease. The back-test compares the reserve with the simulated truth and evaluates the deviance on the lower triangle, neither of which influences any choice in the procedure. Against the three references, the code follows Paper C's network, loss, refit and dispersion, offers the rolling origin of Al-Mudafer *et al.* as its

default validation, and differs in its exact, unsmoothed minimum and in its single seed. Chapter 6 locates each step in the code and checks the procedure numerically.

CHAPTER 6

IMPLEMENTATION AND VERIFICATION

6.1 INTRODUCTION

Chapters 2 to 5 described the procedure as formulas. This chapter maps each step to the function and line of the code that carries it out, and reports a replication of the procedure on synthetic data with which the properties derived in the earlier chapters can be seen at work. It closes with the summary and conclusions of the report.

6.2 WHERE EACH STEP IS IN THE CODE

Table 6.1 lists, for each step of the procedure, where it is implemented.

Table 6.1 Where each step of the procedure is implemented in the code

Step	Where in the code
The three parts of the data, Table 2.1	<code>triangle_sets()</code> in <code>R/triangles.R</code> ; <code>sets\$upper</code> , <code>sets\$train</code> , <code>sets\$vali</code> , <code>sets\$test</code> , <code>sets\$tail_o</code>
The claims split, (2.1) and (2.2)	<code>triangle_sets()</code> : <code>alloc[order(occurrence_period, claim_no),]</code> , <code>alloc\$half <- rep(1:2, length.out = nrow(alloc))</code> , <code>square_of(half == 1L)</code> , <code>stopifnot(all.equal(train_full + vali_full, full))</code>
Masking of zero periods, Section 2.3.3	<code>bccnn_validation()</code> : <code>vali <- mask_zero_periods(upper(vali / scale), odp)</code> ; the helper sets the rows in <code>odp\$zero_origin</code> and the columns in <code>odp\$zero_dev</code> to NA
The training problem (3.1), full batch	<code>fit_bccnn()</code> : <code>fit(x_fit, y_fit, epochs, batch_size = batch_size % % nrow(y_fit))</code> ; <code>compile(loss = "poisson")</code>
RMSprop (3.2)	<code>optimizer_rmsprop(learning_rate, rho, epsilon)</code> ; <code>config.yml bccnn: training:</code>
Seed and initialisation, Section 3.4	<code>bccnn_model()</code> : <code>clear_session()</code> , <code>set_random_seed(seed)</code> ; Keras defaults for <code>layer_dense()</code>
The history, Section 3.2	<code>fit_bccnn()</code> : <code>record()</code> , <code>callback_lambda(on_epoch_end = ...)</code> , <code>keras_poisson_to_deviance()</code>
The validation run and t^* , (4.1) and (4.2)	<code>bccnn_validation()</code> : <code>fit_bccnn(odp, epochs = max_epochs, track = list(vali = vali))</code> , <code>best <- h\$epoch[which.min(h\$vali)]</code>
Override of t^*	<code>bccnn_calibrate()</code> : <code>steps <- if (is.null(epochs)) val\$best_epoch else epochs</code>
The decreases (4.3)	<code>loss_decrease(h, step, window = 10)</code>
The refit, Section 4.3.1	<code>bccnn_calibrate()</code> : <code>odp <- fit_odp_glm(sets\$upper)</code> , <code>nn <- fit_bccnn(odp, epochs = steps, track = list(truth = truth))</code>

Step	Where in the code
The partition alternative, Section 4.3.2	<code>bccnn_calibrate(): mu <- val\$mu_path[[steps + 1]];</code> <code>bccnn_rolling_origin(keep_mu_final = ...)</code>
The dispersion (5.1)	<code>bccnn_phi(odp\$phi, decrease[["vali"]])</code>
The outputs, Section 5.2	<code>bccnn_tables()</code> in <code>R/tables.R</code> ; <code>write_tables()</code> ; the <code>save_gg()</code> calls of the script
The back-test (5.2)	<code>bccnn_tables(): poisson_deviance(test, mu),</code> <code>nn\$reserve - true; add_truth()</code> for the classical fits

6.3 NUMERICAL CHECK

The check script (`bccnn_check.py`, variant with a mild interaction, kept with these explainers) generates a 20×20 portfolio as the sum of two independent over-dispersed Poisson halves with the same effects and intercepts $c - \log 2$, so that the claims split of Section 2.3 is reproduced exactly at the cell level, and then runs the procedure of Sections 4.2 and 4.3 with $T_{\max} = 3000$ to show the whole curve. The data are synthetic and are not a statement about the simulated portfolio of the thesis.

Figure 6.1 shows the early-stopping run of Section 4.2 on the synthetic halves. The top panel shows Keras's reported loss (with dropout, blue) and the deviance without dropout (black) on the training half; the bottom panel shows the validation deviance $D_V(t)$ on the validation half, with t^* marked. The curve has the textbook shape: a fall, a minimum, and over-fitting beyond it.

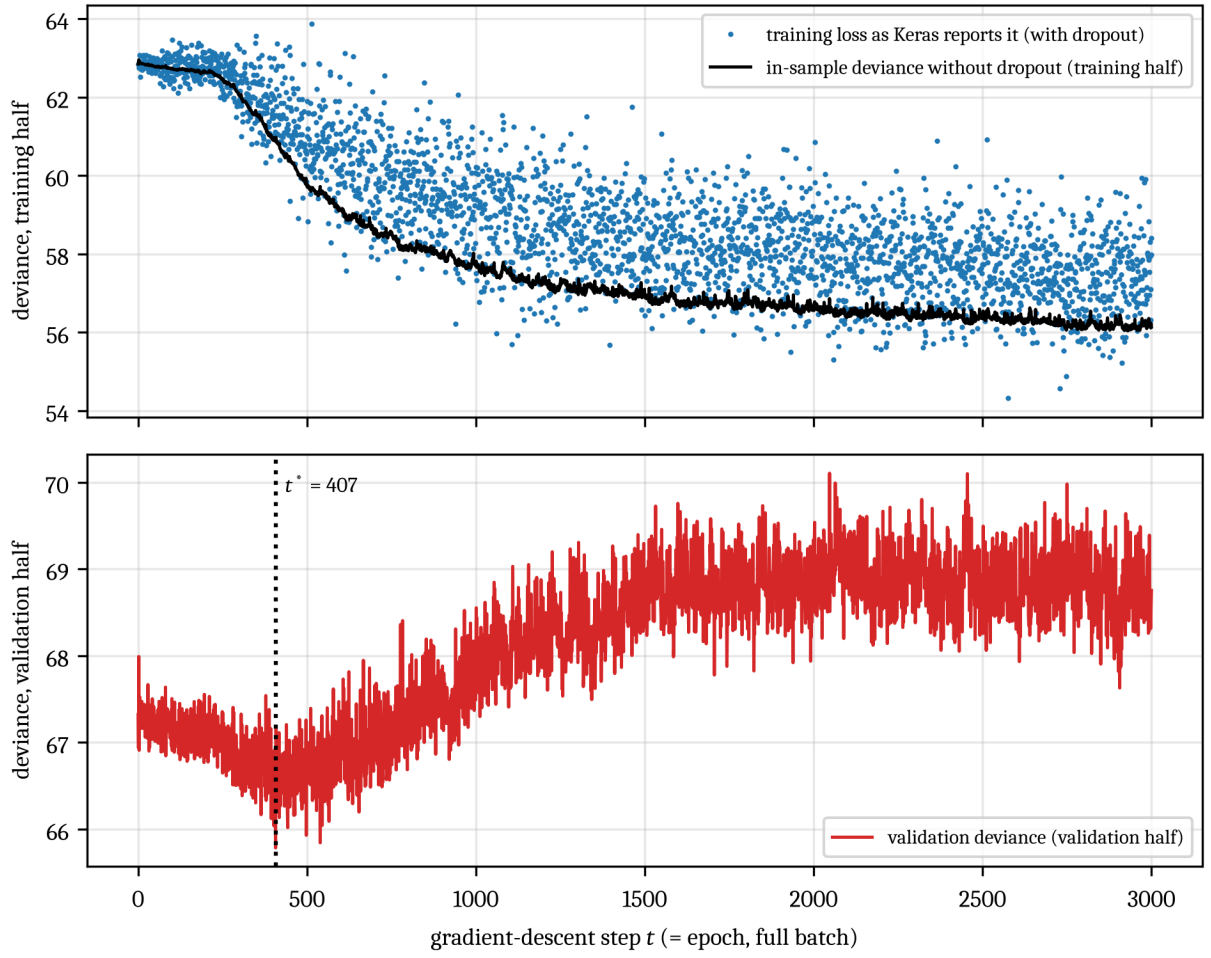


Figure 6.1 The early-stopping run on the synthetic halves

Table 6.2 gives the quantities of the run.

Table 6.2 The procedure on the synthetic halves (amounts in millions of the synthetic data)

Quantity	Value
Intercept of the training half, $\hat{\mathcal{C}}^T$	3.950
Intercept of the full triangle, $\hat{\mathcal{C}}$	4.626
Difference of the intercepts, $\hat{\mathcal{C}} - \hat{\mathcal{C}}^T$	0.676
$\log 2$	0.693
$\max_i \hat{\alpha}_i^T - \hat{\alpha}_i $	0.056
$\max_j \hat{\beta}_j^T - \hat{\beta}_j $	0.413
Run length T_{max}	3 000
t^* by (4.2)	407
$D_V(0)$	67.32
$D_V(t^*)$	65.78

Quantity	Value
$D_V(T_{max})$	68.75
$D_T(0)$	62.84
$D_T(T_{max})$	56.14
Decrease Δ_T at t^* , (4.3)	3.0%
Decrease Δ_T^{drop} at t^* , (4.3)	1.6%
Decrease Δ_V at t^* , (4.3)	2.3%
Chain-ladder reserve $\hat{R}^{cc}$ (refit for $t^* = 407$ steps)	1 227.3
bCCNN reserve $\hat{R}^{bCCNN}$ (refit for $t^* = 407$ steps)	1 218.9
True reserve R	1 438.8
In-sample deviance on O , ccODP	53.7
In-sample deviance on O , bCCNN	52.2
Out-of-sample deviance on F , ccODP	2 506.5
Out-of-sample deviance on F , bCCNN	2 530.7
Dispersion $\hat{\phi}_D$, (5.1)	0.314
Dispersion $\hat{\phi}^{bCCNN} = \hat{\phi}_D(1 - 0.0228)$, (5.1)	0.307

The largest difference of the development effects, 0.413, lies in the sparse last development periods. Both models under-reserve, because the interaction built into the synthetic data grows into the lower triangle.

Two things in Table 6.2 are worth carrying over to the real run. The validation curve reaches its minimum early and then rises for 2 600 steps, so a run of $T_{max} = 1\,000$ would have found the same t^* here, but a patience rule with a short patience would have stopped before step 407 on one of the early dips of the noisy curve; and a 2.3% decrease of the validation deviance translated into a 0.7% move of the reserve, which is the kind of proportion Paper C's Table 2 shows.

6.4 SUMMARY AND CONCLUSIONS

The report has followed the fitting procedure of the bCCNN through the code and written each step as a formula. The observed triangle trains the final model, the two halves of the claims only choose the number of steps, and the true lower triangle only judges the result. Under exchangeable claims the halves share the accident and development effects of the ccODP and differ in the intercept by $\log 2$, so that the validation deviance on one half is directly comparable with the fit on the other; it measures transfer to independent noise at the same cells, not extrapolation into the lower triangle. Full-batch RMSprop bounds every step by 0.00316 per coordinate, so that the number of steps is a hard budget on the distance from the chain ladder, and the exact minimum of the validation deviance over a run of 1 000 steps sets that

budget for the refit. Given the seed, the reported reserve is a deterministic function of the observed triangle, the allocation of the claims, the seed and the fixed hyperparameters.

The numerical check on synthetic halves confirms the relation of the intercepts (0.676 against $\log 2 = 0.693$), shows a validation curve with an interior minimum at step 407 followed by over-fitting, and reproduces the bookkeeping of the tables of the code; a decrease of 2.3% in the validation deviance moved the reserve by 0.7%.

The conclusions are that the transfer of t^* from the validation run to the refit is an assumption supported, under the claims split, by the relation between the halves, and that the least smoothed of the stopping rules in the literature is used in the code. Further work suggested by the procedure is a moving-average minimum on the recorded history in the spirit of Härkönen, a refit with the halves swapped, and a seed study or nagging average over several seeds; each is a change to the script rather than to the procedure.

References

- Al-Mudafer, M.T., Avanzi, B., Taylor, G. & Wong, B. 2021. Stochastic loss reserving with mixture density neural networks. arXiv preprint 2108.07924.
- Department of Statistics and Actuarial Science. 2026. *Research assignment guide*. Last updated 28 January 2026. Stellenbosch: Stellenbosch University.
- Gabrielli, A. 2020. A neural network boosted double overdispersed Poisson claims reserving model. *ASTIN Bulletin*, 50(1):25-60.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2020. Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal*, 2020(1):1-29.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2018. Neural network embedding of the over-dispersed Poisson reserving model. SSRN working paper 3288454, version of 21 November 2018.
- Goodfellow, I., Bengio, Y. & Courville, A. 2016. *Deep learning*. Cambridge, MA: MIT Press.
- Härkönen, V. 2021. On claims reserving with machine learning techniques. Unpublished master's thesis. Stockholm: Stockholm University.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. 2014. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929-1958.
- Wüthrich, M.V. & Merz, M. 2019. Editorial: yes, we CANN! *ASTIN Bulletin*, 49(1):1-3.

APPENDIX A

LIST OF SYMBOLS

The symbols are those of companion I; the rows below the separating row are new in this report.

Table A.1 Symbols used in this report

Symbol	Meaning	Section
n	Accident periods = development periods, $n = 20$	2.2
i, j, k	Accident period, development period (both $1, \dots, n$), calendar period $k = i + j - 1$	2.2
O, F	Observed cells ($i + j \leq n + 1$, 210 of them) and future cells (190)	2.2
$Y_{i,j}, y_{i,j}$	Incremental payments of a cell, in millions	2.2
$c, \alpha_i, \beta_j; \hat{c}, \hat{\alpha}, \hat{\beta}$	ccODP parameters and their MLE	2.3
$D(y, \mu; S)$	Poisson deviance on the cells S , (2.4) of companion I	3.2
θ, θ_0	The 547 trainable parameters; the start of gradient descent, (3.7) of companion I	3.2
$\mu_{i,j}(\theta)$	The mean of the network, (3.5) of companion I	3.2
$\ell(\theta)$	The loss Keras minimises, (3.8) of companion I	3.2
η, ρ, ϵ	RMSprop learning rate, decay, stabiliser: 0.001, 0.9, 10^{-7}	3.3
$\hat{R}$	Reserve, the sum of the means over F	5.2
	<i>New in this report</i>	
D	The observed triangle of the whole portfolio (sets\$upper), the training data of the final fit	2.2
D^T, D^V	Observed triangles of the training half and of the validation half of the claims (sets\$train, sets\$vali)	2.3
$y_{i,j}^T, y_{i,j}^V$	Their cells	2.3
T	The test set: the true lower triangle (sets\$test)	2.2, 5.3
N	Number of cells the network is trained on: 210 for D and for D^T	3.2
s	The seed (cfg\$seed = 2026)	3.4

Symbol	Meaning	Section
t, θ_t	Gradient-descent step $0, 1, 2, \dots$ (= Keras epoch, full batch); the parameters after step t	3.2
T_{max}	Length of the early-stopping run, max_epochs = 1000	4.2
$D_T(t), D_V(t)$	Deviance of $\mu(\theta_t)$ on the training cells and on the validation cells, without dropout	4.2
ℓ_t^{drop}	The loss Keras reports for epoch t (with dropout, before the update of that epoch)	4.2
t^*	The step with the lowest validation deviance; the number of steps of the final fit	4.2
$\Delta_T, \Delta_V, \Delta_T^{drop}$	Relative decreases of the losses from the start to t^* (Paper C's Table 3)	4.2.4
v_t	The running mean of squared gradients of RMSprop	3.3
$T_t^s(\theta, S)$	The map ' t RMSprop steps with seed s on the cells S , from θ '	4.4